

프로젝트 최종 발표
2024. 06. 10



커넥티드 플랫폼의 이론과 실제 Term Project

WebOS기반의 Media Web Application

1조

2019147002 최진호

2019147026 김현중

2019147029 조은기

2020147542 강슬미



목차

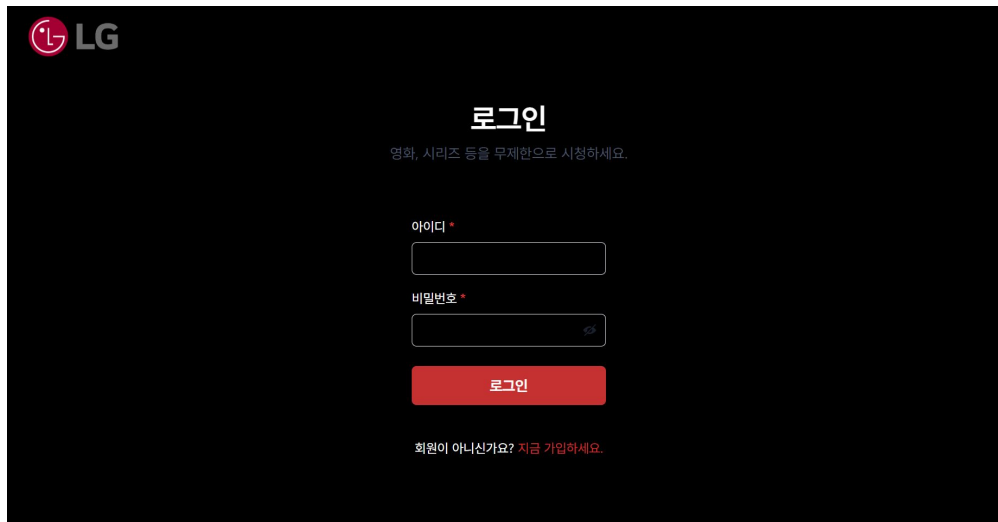
1. Introduction
2. Architecture
3. Demo
4. Decisions
5. Lessons Learned



1. Introduction

1-1. Project Objective

- 넷플릭스, 디즈니플러스와 같은 **Media Web Application**을 제작
- **UX**(User Experience)와 **DX**(Developer Experience)를 동시에 고려하는, 높은 퀄리티의 어플리케이션을 제작
- 스마트TV가 갖는 **Family Device**로서의 역할을 극대화 ex) 멀티 프로필 기능





1-2. Roles

- 최진호(2019147002): client 개발, UI/UX 디자인
- 김현중(2019147026): server 개발, DevOps, QA
- 조은기(2019147029): server 개발, DevOps, merge 전 Test Code 검수
- 강슬미(2020147542): server 개발, DBA



1-3. Workflows

1. Figma를 통한
화면 및 유저
스토리 정의

2. GitHub
Discussion을
통한 API 및 DB
설계

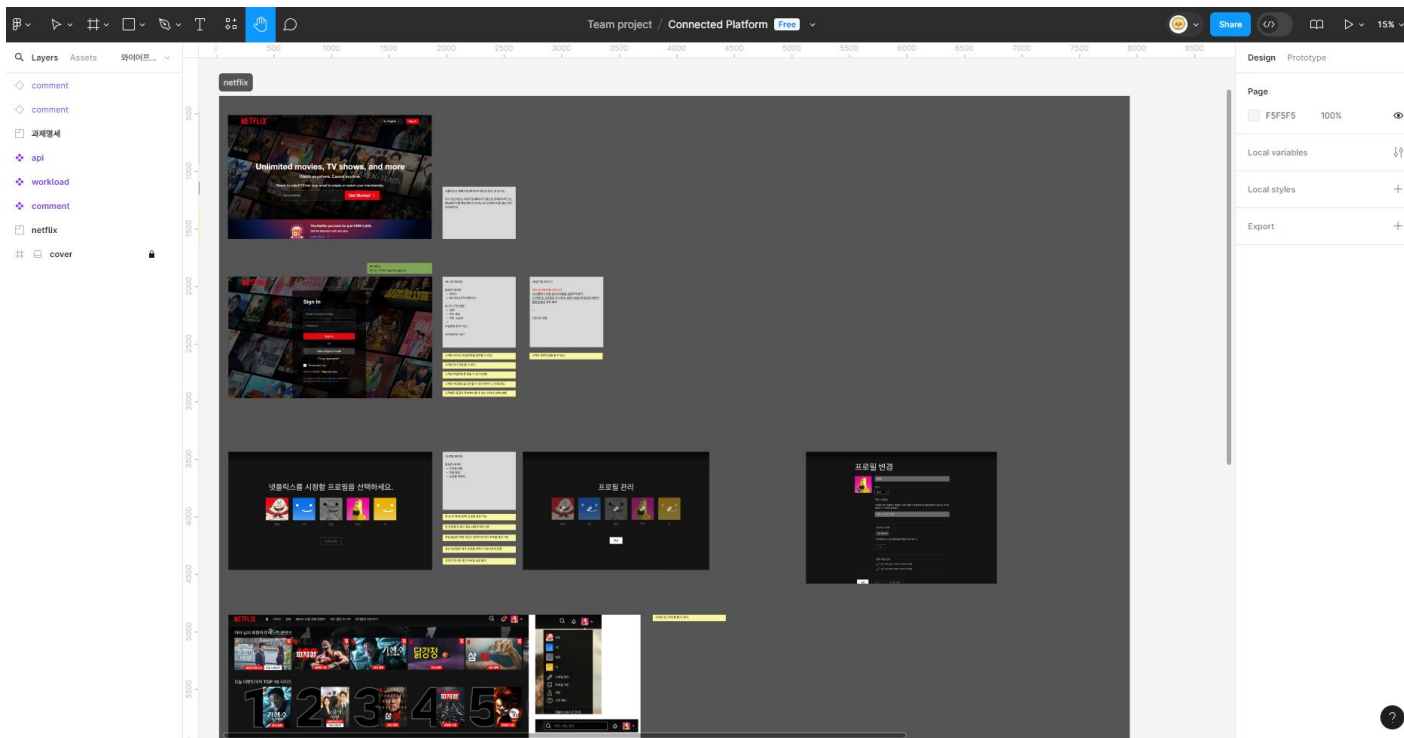
3. Github
Issue를 통한
이슈레이징 및
일정산정

4. GitHub Pull
Request를 통한
코드리뷰 및 반영

위 4가지 step의 iteration을 통해 보다 빠르고 엄밀하게
요구사항 분석(Requirements Engineering) 및 프로젝트 관리(Project Management)



1-3. Workflows





1-3. Workflows

☐	☒	feat: 비디오 플레이어 시간 정보 연동하기	client	feature	1	
#63 by jinoov was closed 3 weeks ago 2 tasks done						
☐	☒	feat: 영상 재생 플레이어 추가	client	feature	1	
#59 by jinoov was closed 3 weeks ago 2 tasks done						
☐	☒	feat: 영상 상세페이지 UI 추가	client	feature	1	
#58 by jinoov was closed 3 weeks ago 2 tasks done						
☐	☒	feat: 영상 재생 페이지 UI 추가	client	feature	1	
#57 by jinoov was closed 3 weeks ago 2 tasks done						
☐	☒	feat: 어디까지 시청했는지 기록		feature	1	
#55 by 2020147542 was closed last week 2 tasks done						
☐	☒	feat: 유저별 프로필 리스트 조회		feature		
#54 by 2020147542 was closed last week 2 tasks done						
☐	☒	feat: 프로필 별 시청기록 조회		feature		
#53 by 2020147542 was closed last week 2 tasks done						
☐	☒	feat: 프로필에서 빠져나가기		feature	1	
#52 by glitchann was closed last week 2 tasks						
☐	☒	feat: 개인정보 조회		feature	1	
#50 by glitchann was closed 2 weeks ago 2 tasks done						
☐	☒	feat: 로그인하기		feature	1	
#49 by glitchann was closed 2 weeks ago 2 tasks done						
☐	☒	feat: 프로필 생성 기능		feature		
#48 by 2020147542 was closed last week 2 tasks done						
☐	☒	test: 회원가입 테스트 코드 작성		feature		1
#47 by glitchann was closed 2 weeks ago 2 tasks done						
☐	☒	feat: 인스타그램 코드 작성		feature	1	
#45 by glitchann was closed 3 weeks ago 2 tasks done						

Github Issue

Merged

[server] feat: 프로필별 시청기록 조회 기능 구현 #77
jinoov merged 2 commits into [main](#) from [feature/53](#) 2 weeks ago

69 +
70 +
71 +

// 로그인 확인 @Session
List<Long> contentsIdSet = profilecontentsrepository.findByProfileIdAndStateNot(profileId, Pro

jinoov 3 weeks ago • edited

'프로필별 시청기록 조회'라고 이슈이름에 지어져있었으나 조금 혼동의 여지가 있어보여요.
해당 api의 목적은 시청중인 콘텐츠를 조회하는 것입니다. (저희 페이지 중에 자기 시청기록을 조회하는 페이지는 없어요)
그래서 해당 라인이 이연식으로 수정되어야할 것 같습니다
`List<Long> contentsIdSet = profilecontentsrepository.findByProfileIdAndState(profileId, ProfileContents.State.WATCHING)`
혹시 제가 잘못 이해한 게 있다면 말씀해주세요!
2020147542 2 weeks ago
이 제가 잘못 이해했었네요. 수정하겠습니다
Reply...

Unresolve conversation jinoov marked this conversation as resolved.

[server] refactor: 시청중인 콘텐츠만 불러오기로 변경 9876624

2020147542 commented 2 weeks ago
change request 반영했습니다!
1



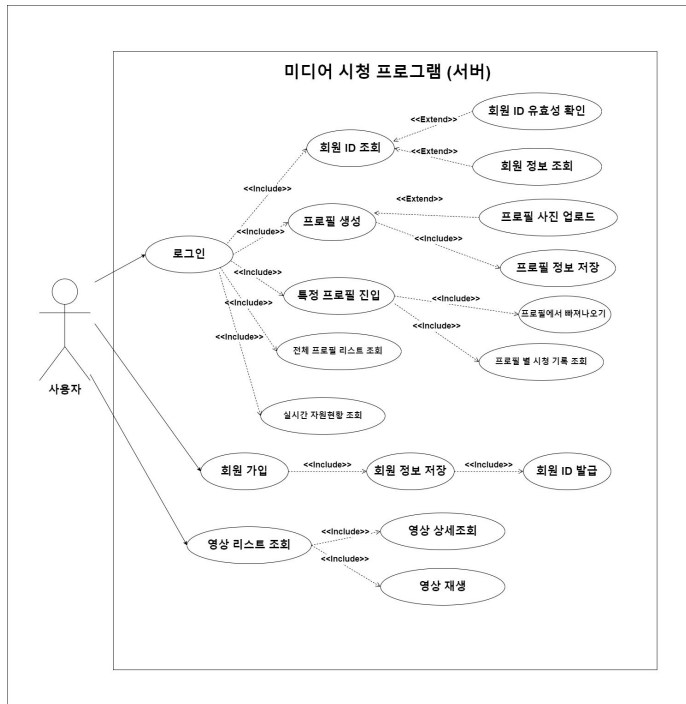
1-3. Workflows

The screenshot shows a Slack interface for a workspace named 'webos-yonseoi'. The left sidebar contains navigation options: Home, DMs, Activity, and More. The main channel is '# 깃허브-알림'. A notification from the GitHub app is displayed, dated Thursday, June 6th, 9:38 PM. The notification is for a pull request opened by user 2020147542. The message content includes: '#188 [all] docs: 테스트 계획 및 결과 문서 초안 작성', 'Major Changes' with a bulleted list of test plan and review requirements, 'Related Issues' mentioning issue #187, a checkmark indicating 'please merge in Squash & Merge!', reviewers '@gitchannn, @jinoov', and labels 'docs, all'. Below the notification, a commit message is visible: '1 new commit pushed to main by jinoov' with a commit hash '94170643'. The commit message text is: '94170643 : [all] docs: 테스트 계획 및 결과 문서 초안 작성 (#188)'. The bottom of the screen shows a text input area for the channel with a message placeholder 'Message #깃허브-알림' and various formatting icons.

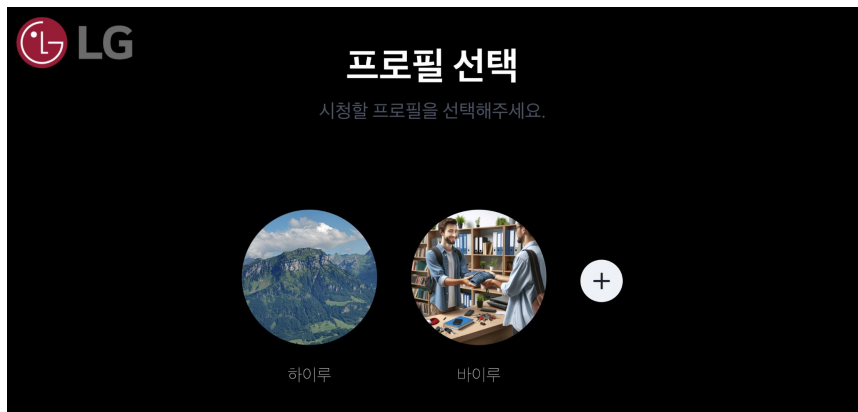
1-4. Requirements

필수기능

- 고객은 실시간 자원현황(CPU, Memory)을 시각화해서 볼 수 있다.
- 고객은 미디어 재생목록을 조회할 수 있다.
- 고객은 미디어를 재생할 수 있다.
- 고객은 미디어를 마지막 정지 시점부터 이어서 볼 수 있다.
- 고객은 특정 아이디를 이용해서 식별될 수 있다.



1-4. Requirements



심화기능

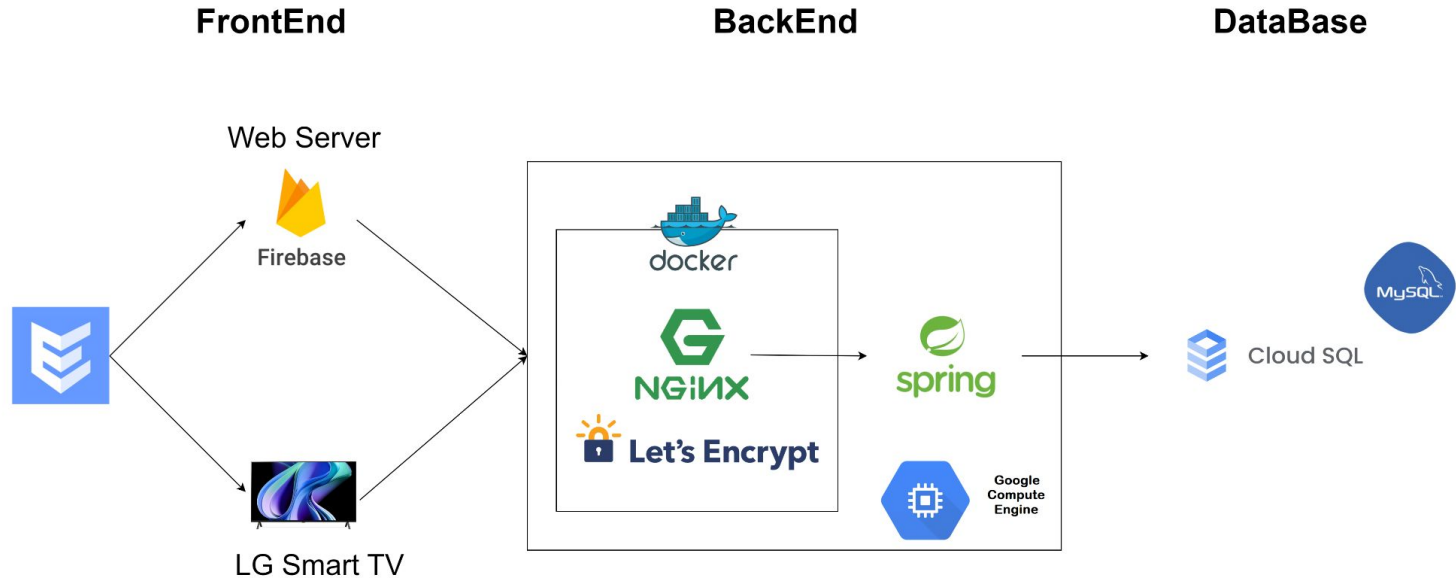
- 고객은 멀티프로필을 생성할 수 있다.
- 고객은 특정 프로필을 하나의 플랫폼으로만 접속 가능하다.
- 고객은 성인여부에 따라 시청가능한 콘텐츠가 다르다.
- 고객은 실시간 어플리케이션 동작 현황을 조회할 수 있다.
- 고객은 Adaptive Streaming 기능을 사용할 수 있다.



2. Architecture



2-1. Architecture Overview





2-1. Architecture Overview

FrontEnd

- Enact 기반 개발
- Web(Google Firebase), Embedded(LG Smart TV)에 배포

BackEnd

- Spring Boot 기반 개발
- Google Compute Engine 환경에 배포
- Docker를 이용해 Nginx와 HTTPS 환경 구축

DataBase

- Google Cloud SQL에서 MySQL 환경 구축

2-2. Tech Stacks

FrontEnd

- Enact 베이스 
- TypeScript를 전 범위 코드에 적용해서 컴파일타임 안정성 확보 
- Tanstack Router를 이용한 라우팅 처리 및 데이터 로딩 
- Zustand를 이용한 micro state management 

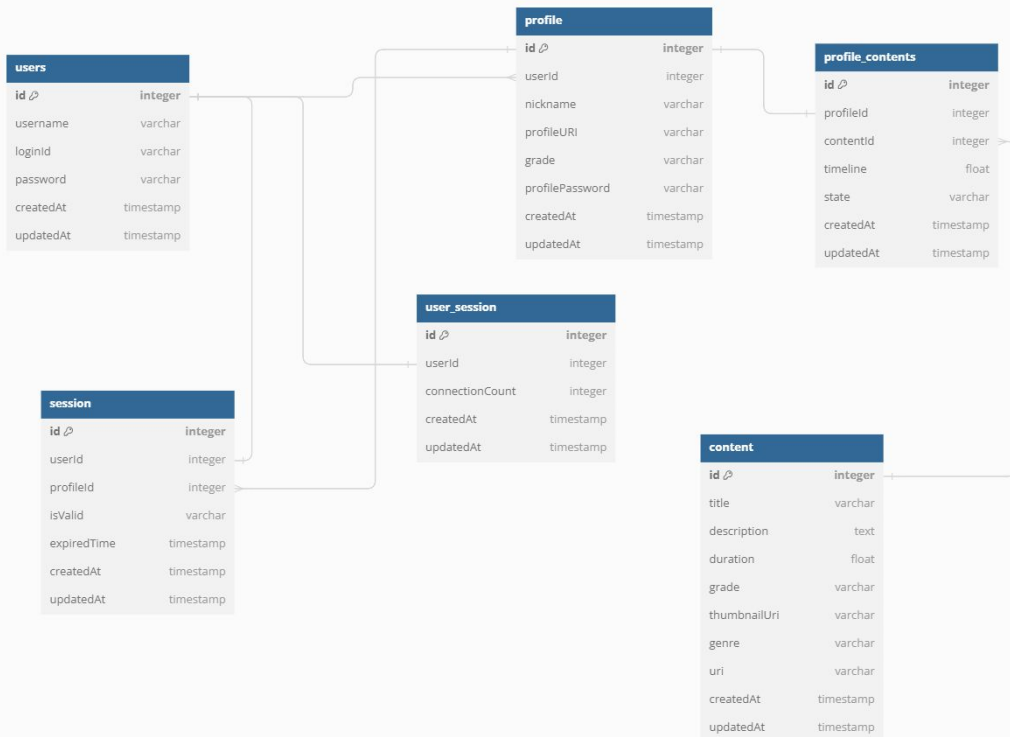
2-2. Tech Stacks

BackEnd

- Spring Boot 베이스 
- Spring Data JPA를 활용하여 생산성 및 안정성 확보 
- JUnit을 활용하여, 사용자 시나리오를 따르는 인수테스트 수행 **J**Unit
- MySQL을 활용한 영속성 데이터 저장 



2-3. Data-Model Design



MySQL 선택 이유

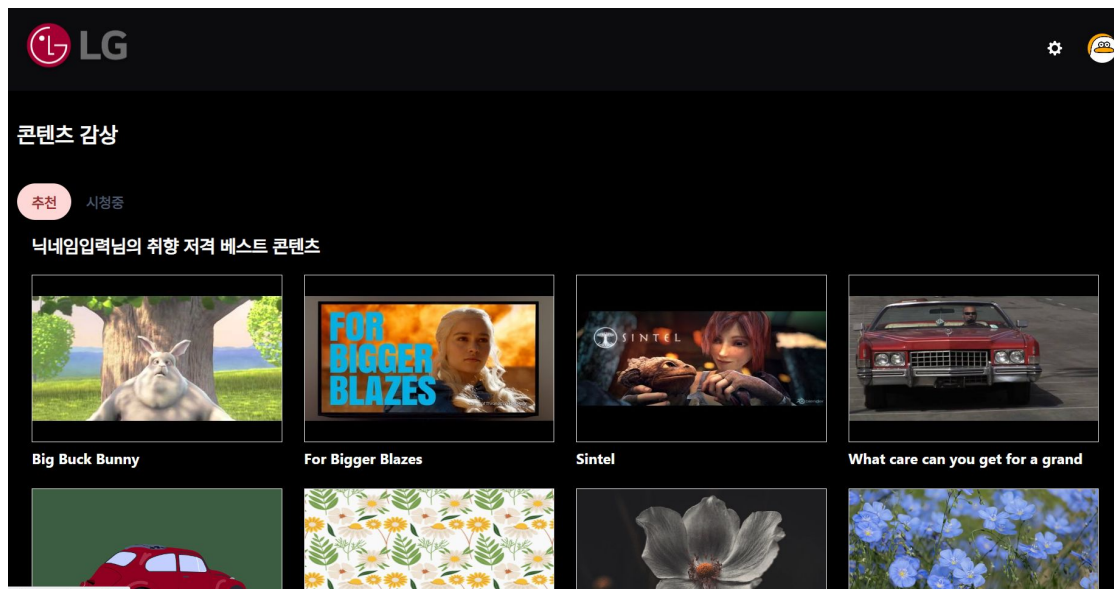
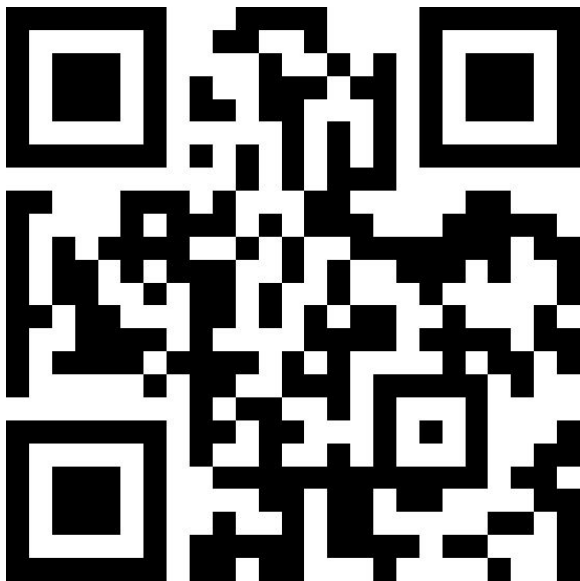
- 안정성과 성능도가 높고, 광범위한 커뮤니티 지원으로 문제 해결이 용이함
- 프로젝트의 Information Architecture가 관계형 데이터 모델에 적합
- 다양한 플랫폼과의 높은 호환성으로 유연한 개발 가능



3. Demo

3-1. Project Demo

서비스 URL: <https://webos-yonsei.web.app/>





4. Decisions



4-1. FrontEnd Decisions

프로젝트 빌드 타임 개선

@enact/cli는 Create-React-App(이하 CRA)을 기반으로 만들어진 템플릿

그러나 CRA는 유지보수가 거의 되지 않고 있으며, 다른 템플릿 대비 빌드타임이 느리다는 한계가 존재한다.

➔ **enact** 템플릿 내부를 뜯어고치자!

Option 1: Create a new framework from scratch

We could try to rearchitect Create React App as a framework that integrates data fetching, routing, bundling, and SSG/SSR. Building a high-quality new framework at the intersection of these concerns is a huge undertaking, requires a lot of specialized ecosystem expertise, and even if we stopped other projects to pull this off, there is a significant risk it would become stagnant over time like Create React App itself did. It would also further fragment the ecosystem with yet another framework that is officially recommended despite having no real users. We don't think this option is practical at this time.

Option 2: Deprecate Create React App, maintain a Vite template

We could deprecate Create React App and instead maintain our own Vite template. To achieve the stated goals, this template would have to be very sophisticated. In fact, it would have to be as sophisticated as a React framework — and impose opinions about routing, data fetching, etc. That leads to the same issue: we'd effectively be creating another framework.

Option 3: Deprecate Create React App, suggest React frameworks

We could deemphasize or deprecate Create React App as a tool, and emphasize React frameworks more actively. This doesn't mean you would *have* to use a framework with React, but that we would *suggest* using one of them for most apps. The downside is we'd no longer have a neutrally branded CLI "gateway" to creating a React app: you'd have to find the right one in the corresponding framework's docs. Outright deprecating is also disruptive. We'd need to keep the command working for a long while — which is confusing from the branding perspective ("why is creating a React app deprecated?")

[react/reactjs-dev] Replace Create React App recommendation with Vite



- `npm run eject` 실행을 통해 템플릿 해체
- 더이상 유지보수 되지않는 플러그인들을 삭제
- 더 빠른 성능의 플러그인으로 교체



4-1. FrontEnd Decisions

프로젝트 빌드 타임 개선

- CJS + JavaScript 기반의 babel-loader를
ESM + Golang 기반의 esbuild-loader로 교체
→ **빌드시 70% 이상의 속도 향상**
- 패키지매니저를 npm에서 bun으로 교체 →
패키지 초기 설치시 4배 정도의 빠른 속도





4-1. FrontEnd Decisions

프로젝트 빌드 타임 개선

(좌)

아무것도 세팅되어있지 않은
초기프로젝트 ➡ 14.74 secs

(우)

수많은 기능들이 추가된 현재 프로젝트 ➡
4.82 secs

빌드속도가 70% 가량 더 빠름

CSI4131.01-00

```
~/work/lets-eject-enact main !6 75
> bun pack -p
$ node ./scripts/pack.js -p
Creating an optimized production build...
(node:61948) [DEP_WEBPACK_COMPILATION_NORMAL_MODULE_NOT_FOUND_PLUGIN]
o NormalModule.getCompilationHooks(compilation).
(Use `node --trace-deprecation ...` to show where the warning was moved)

SMP ●
General output time took 14.74 secs

SMP ● Plugins
IlibPlugin took 8.88 secs
TerserPlugin took 4.14 secs
HtmlWebpackPlugin took 3.38 secs
ESLintWebpackPlugin took 2.16 secs
CaseSensitivePathsPlugin took 1.55 secs
CssMinimizerPlugin took 0.84 secs
WebOSMetaPlugin took 0.013 secs
DefinePlugin took 0.002 secs
GracefulFsPlugin took 0 secs
ModuleNotFoundPlugin took 0 secs
EnvironmentPlugin took 0 secs

SMP ● Loaders
mini-css-extract-plugin, and
css-loader, and
postcss-loader, and
less-loader took 3.21 secs
  module count = 2
css-loader, and
postcss-loader, and
less-loader took 2.7 secs
  module count = 2
mini-css-extract-plugin, and
css-loader, and
postcss-loader took 2.65 secs
  module count = 19
modules with no loaders took 2.47 secs
  module count = 731
css-loader, and
```

```
~/work/WebOS-yonsei/client main
> bun run pack-p
$ node ./scripts/pack.js -p
Creating an optimized production build...
(node:33891) [DEP_WEBPACK_COMPILATION_NORMAL_MODULE_NOT_FOUND_PLUGIN]
r was moved to NormalModule.getCompilationHo
(Use `node --trace-deprecation ...` to show where the warning was moved)

SMP ●
General output time took 4.82 secs

SMP ● Plugins
TerserPlugin took 0.514 secs
HtmlWebpackPlugin took 0.145 secs
WebOSMetaPlugin took 0.012 secs
ForkTsCheckerWebpackPlugin took 0.002 secs
DefinePlugin took 0.002 secs
GracefulFsPlugin took 0.001 secs
EnvironmentPlugin took 0.001 secs
CssMinimizerPlugin took 0.001 secs
ModuleNotFoundPlugin took 0 secs

SMP ● Loaders
modules with no loaders took 2.57 secs
  module count = 1667
esbuild-loader took 0.72 secs
  module count = 79
html-webpack-plugin took 0.01 secs
  module count = 1

Compiled successfully.

1.84 MB dist/main.js
```


4-1. BackEnd Decisions

1계정당 최대 3명 동시접속 기능

[기존 구현]

로그인 요청시, 테이블(session) 속 해당 아이디의 유효한 세션 개수를 세기
유효한 세션 개수가 3개 이상인 경우, 로그인 요청을 거부

➡ **매번 전체 테이블을 순회해야하기 때문에 비효율이 발생**

session	
123	id
123	is_valid
123	profile_id
123	user_id

4-1. BackEnd Decisions

1계정당 최대 3명 동시접속 기능

[개선안]

아이디별로, 유효세션의 수를 기록하는 별도 테이블(user_sessions)을 생성

➔ **$O(N)$ 의 시간복잡도를 $O(1)$ 로 감소!**

비정규화 과정으로 인해 데이터 무결성 관리의 cost는 늘었지만,

성능 최적화의 benefit이 더 크다고 판단하였음

users_session	
123	id
123	connection_count
123	user_id



5. Lessons

5-1. 좋은 협업 방식에 대한 고민

문제

- 코드 컨벤션 및 GitHub 사용에
구성원들간 서로 다른 이해
- 공통 코드의 다른 구현부 병합 시
기능 훼손 방지 어려움

해결

자주 병합하기

- 기초적인 빌드 여부 및 테스트
통과 여부를 확인하는 CI 도입
- 자주 병합을 시도하여 코드 충돌
및 오류를 사전 예방

테스트 환경 개선

- 숙련도 차이를 고려하여 시나리오
기반 인수 테스트를 필수 테스트로
선정
- 테스트 환경을 격리하는 템플릿
제작
- 안정적이고 쾌적한 테스트 환경
제공

효과

- 전체 팀원의 협업 효율성 증가
- 코드 병합 시 기능 훼손 최소화
- 테스트 과정의 안정성 및 비용
효율성 향상

5-2. 임베디드 환경에 대한 고민

문제

- 임베디드 크롬의 버전이 데스크탑의 버전과 상이
- 로컬 개발 환경과 프로덕션 환경에서의 CSS, JS 등의 적용 모습이 달랐음

해결

호환성을 고려한 CSS 작성

- 버전별 호환성 꾸준히 체크
- 호환이 안되는 CSS 문법 지양

JavaScript 트랜스파일링

- 이미 번들링한 결과물을 ES2015를 타겟팅해서 다시 트랜스파일링 임베디드 크롬에서도 JS가 정상동작할 수 있게 함

효과

- 로컬 개발환경과 프로덕션 환경에서의 동일한 작동 보장 가능
- 스마트TV에 최적화된 어플리케이션 개발

Q & A

감사합니다.

